MISP objects and templates in 2.5

Model observations, validate instances, contribute templates

CIRCL / MISP Project

https://www.misp-project.org/

Reference snapshot: 29 October 2026



Model meaning, not just a bag of values

- Choose an existing model before inventing one.
- Separate template identity from instance identity.
- Validate definitions, instances and sharing policy.
- Practise an offline example and a contribution workflow.

Model meaning, not just a bag of values
- Choose an existing model before inventing one.
- Separate template identity from instance identity.
- Validate definitions, instances and sharing policy.
- Practise an offline example and a contribution workflow.

Allow 35–45 minutes plus a 25–35 minute workshop. Reference MISP 2.5.50 and the pinned misp-objects checkout reviewed on 29 October 2026. Availability varies with installed templates and patch level. The README contains the source audit and precise commands.

Understand the model

Attribute, object and template

- An attribute is a typed observation with its own UUID.
- An object groups attributes describing one entity or observation.
- A template describes relation names, types and constraints.

Attribute, object and template
- An attribute is a typed observation with its own UUID.
- An object groups attributes describing one entity or observation.
- A template describes relation names, types and constraints.

A template is a model definition, not an event or object instance. The same template can describe many instances, each with a different UUID. Object grouping should preserve a coherent source and time context rather than mixing unrelated observations merely because a tool emitted them together.

Three identifiers with different jobs

- `uuid`: the object instance.
- `template_uuid`: the reusable model.
- `template_version`: the revision used to construct it.

Three identifiers with different jobs
- uuid: the object instance.
- template_uuid: the reusable model.
- template_version: the revision used to construct it.

Attributes also have UUIDs. References point to instance UUIDs, not the template UUID. Keep the recorded template revision in exports and do not assume a newer locally installed definition rewrites existing object data. Numeric template versions can appear as strings in serialised client output.

Reuse the current repository

- Inspect the actual definition, not just its name.
- Look under `objects/<name>/definition.json`.
- Compare the model with your source data and downstream consumers.

Reuse the current repository
- Inspect the actual definition, not just its name.
- Look under objects/<name>/definition.json.
- Compare the model with your source data and downstream consumers.

The audited repository contains 454 definitions, spanning network, file, financial, vulnerability, detection and threat-hunting contexts. Counts are snapshots. The domain-ip definition is version 11, whereas an illustrative README example still shows version 8: the actual definition and schema are the implementation references.

Current domain-ip: alternatives, not a mandatory pair

- `requiredOneOf`: ip, domain or hostname.
- Repeated domain and IP fields are supported.
- Use separate instances for different observation contexts.

Current domain-ip: alternatives, not a mandatory pair
- `requiredOneOf`: ip, domain or hostname.
- Repeated domain and IP fields are supported.
- Use separate instances for different observation contexts.

The current definition no longer requires both domain and IP. The workshop uses one domain and two IPs to demonstrate repetition, not because all three are mandatory. Inspect first-seen/last-seen and other optional relations before adding a custom field. A relation named ip maps to the existing MISP type ip-dst.

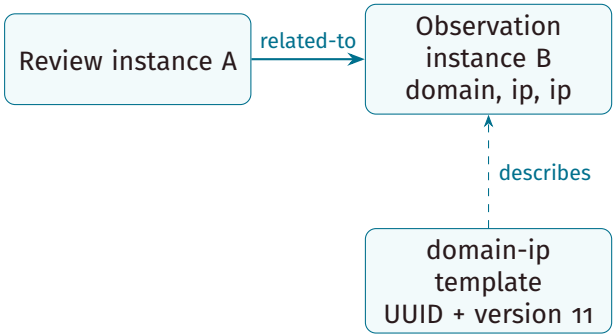
Field relation is not a graph relationship

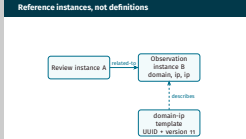
- `ip`: a field relation inside an object.
- `ip-dst`: the underlying MISP attribute type.
- `related-to`: a directed reference between instances.

Field relation is not a graph relationship
- ip: a field relation inside an object.
- ip-dst: the underlying MISP attribute type.
- related-to: a directed reference between instances.

ObjectReference, value correlation and analyst-data relationships are separate mechanisms. A generic related-to edge does not prove causality, and adding an edge does not automatically create an inverse reference. The relationships catalogue describes names, meanings and optional opposites.

Reference instances, not definitions





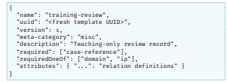
The stored reference targets the UUID of instance B. The template UUID identifies the reusable model and cannot stand in for B. The dashed line is a conceptual modelling link, not an ObjectReference emitted by the exercise. Reference direction is deliberate; no inverse is automatically created.

Build and validate

A definition has its own identity

```
{
  "name": "training-review",
  "uuid": "<fresh template UUID>",
  "version": 1,
  "meta-category": "misc",
  "description": "Teaching-only review record",
  "required": ["case-reference"],
  "requiredOneOf": ["domain", "ip"],
  "attributes": { "...": "relation definitions" }
}
```


A definition has its own identity



Structural illustration only: the angle-bracket UUID and abbreviated attributes are not executable JSON definitions. The complete schema-valid teaching template is in examples/training-review.json. A new model needs a fresh UUID; an edit to an existing model keeps its UUID and increments version.

A relation maps meaning to an existing type

```
"status": {
  "description": "Review status.",
  "misp-attribute": "text",
  "ui-priority": 1,
  "values_list": ["pending", "reviewed"],
  "to_ids": false,
  "disable_correlation": true
}
```


A relation maps meaning to an existing type

"status": {
  "description": "Review status.",
  "misp-attribute": "text",
  "ui-priority": 1,
  "values_list": ["pending", "reviewed"],
  "to_ids": false,
  "disable_correlation": true
}

This is a relation fragment from the teaching definition. Each relation requires description, misp-attribute and ui-priority. Types and meta-categories are enumerated in the current schema. JSON syntax validation alone does not prove that a new field has a useful or compatible meaning.

Required fields and repeated fields

- Every relation in required must be present.
- At least one relation in requiredOneOf must be present.
- Both rules can apply together; multiple permits repetition.

Required fields and repeated fields
- Every relation in required must be present.
- At least one relation in requiredOneOf must be present.
- Both rules can apply together; multiple permits repetition.

The training review needs a case reference AND at least one domain or IP. A second case-reference is invalid because that relation is not repeatable; several domains or IPs are allowed. Requirement lists refer to relation names, not MISP attribute types. Check that every named requirement exists in attributes.

Vocabulary and suggestions are different

- `values_list` drives a vocabulary selector.
- `sane_default` offers suggestions plus manual input.
- Validate vocabulary membership explicitly when your workflow requires it.

Vocabulary and suggestions are different
- values_list drives a vocabulary selector.
- sane_default offers suggestions plus manual input.
- Validate vocabulary membership explicitly when your workflow requires it.

The correct key is singular sane_default. The schema rejects sane_defaults. UI selection controls and server/client validation are different layers. Do not claim PyMISP strict validation universally enforces every vocabulary or application policy. The UI can encounter pre-existing values outside its list.

UI priority is implemented in current MISP

- Higher-priority relations are ordered first in current object views.
- Low-priority fields can keep secondary details out of the main focus.
- Priority is presentation metadata, not a requirement or access rule.

UI priority is implemented in current MISP
- Higher-priority relations are ordered first in current object views.
- Low-priority fields can keep secondary details out of the main focus.
- Priority is presentation metadata, not a requirement or access rule.

The old lesson described ui-priority as future work. Current MispObject fieldOrderByTemplate uses the installed template definitions and sorts priorities descending. Specific themes and views can differ. Recommended is also optional metadata; its presence does not guarantee identical rendering in every consumer.

Detection, correlation and visibility are separate

- Choose `to_ids` from the observation and deployment purpose.
- Choose correlation behaviour from the meaning of the value.
- Review event/object/attribute distribution and sharing context.

Detection, correlation and visibility are separate
- Choose to_ids from the observation and deployment purpose.
- Choose correlation behaviour from the meaning of the value.
- Review event/object/attribute distribution and sharing context.

A template default is not a sharing policy or a guarantee of every resulting flag. The offline example explicitly passes to_ids=False for each added attribute, since the official template may override a broad client default. The custom review disables correlation for local case references and annotations. Event distribution zero keeps the synthetic event organisation-only.

Four validation layers

- Definition schema: recognised fields and types.
- Definition semantics: identity, requirements and compatibility.
- Instance conformity: required and repeatable relations.
- Attribute validity and policy: values, flags and permissions.

Four validation layers
- Definition schema: recognised fields and types.
- Definition semantics: identity, requirements and compatibility.
- Instance conformity: required and repeatable relations.
- Attribute validity and policy: values, flags and permissions.

The schema is not a complete semantic validator: its UUID property is a string, for example. The exercise separately checks UUID syntax/uniqueness and defined requirement names. The full upstream validator adds repository formatting, relationships tests and uniqueness checks. Valid JSON alone does not establish interoperability.

Template enforcement depends on the path

- Template-backed object creation checks conformity.
- Event capture, editing and synchronisation have different paths.
- Strict client construction is useful evidence, not universal server enforcement.

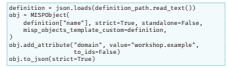
Template enforcement depends on the path
- Template-backed object creation checks conformity.
- Event capture, editing and synchronisation have different paths.
- Strict client construction is useful evidence, not universal server enforcement.

The source audit checks ObjectsController::add and ObjectTemplate::checkTemplateConformity. The latter checks required, requiredOneOf and repetition, while attribute validation happens separately. Avoid the old blanket statement that every rule is enforced through every import. Test the exact API/UI path your integration uses.

Load the selected definition explicitly

```
definition = json.loads(definition_path.read_text())
obj = MISPObject(
    definition["name"], strict=True, standalone=False,
    misp_objects_template_custom=definition,
)
obj.add_attribute("domain", value="workshop.example",
                  to_ids=False)
obj.to_json(strict=True)
```


Load the selected definition explicitly



The real builder loads the official domain-ip definition from the supplied checkout, rather than silently using a bundled PyMISP revision. Strict serialisation checks requirements and repeated unique relations. This excerpt assumes imports and a suitable definition_path; the complete executable file is provided in examples.

The offline workshop event

- One domain/IP object: one domain, two IPs.
- One teaching review: case reference, domain and review status.
- A related-to edge points from review to observation.

The offline workshop event
- One domain/IP object: one domain, two IPs.
- One teaching review: case reference, domain and review status.
- A related-to edge points from review to observation.

The builder generates fresh instance UUIDs on each run and uses reserved .example domains and documentation IPs. Both objects are validated before being attached to an unpublished organisation-only MISPEvent. standalone=False preserves references in the containing event. The output is bare event JSON, not an Event response envelope.

Validate, then inspect the output

```
python examples/verify_examples.py /tmp/misp-objects-lab
python examples/build_examples.py /tmp/misp-objects-lab \
  > training-event.json
```


Validate, then inspect the output
python examples/verify_examples.py /tmp/misp-objects-lab
python examples/build_examples.py /tmp/misp-objects-lab \
> training-event.json

Install the pinned requirements in a virtual environment and check out the reviewed misp-objects revision first; full commands are in the README. No API key or network calls are used by these scripts. Inspect object/template UUIDs, version, repeated relations, references and flags rather than treating successful serialisation as sufficient review.

Make failure cases part of the exercise

- Remove the required case reference.
- Remove all requiredOneOf alternatives.
- Repeat a unique relation or add an unknown relation.
- Misspell a schema field and observe the schema failure.

Make failure cases part of the exercise
- Remove the required case reference.
- Remove all requiredOneOf alternatives.
- Repeat a unique relation or add an unknown relation.
- Misspell a schema field and observe the schema failure.

The verifier exercises five negative cases using the real PyMISP and jsonschema libraries. Unknown-relation construction may fail as an attribute-type error rather than a conformity exception. It also verifies JSON round-trip, instance-reference targets, identity and explicit detection flags. These are offline checks, not HTTP/server tests.

Deploy and contribute

Disk definitions and installed templates are separate

- Update the selected Git checkout deliberately.
- Import/update the template database through the operator workflow.
- Confirm the intended UUID and revision are available to users.

Disk definitions and installed templates are separate
- Update the selected Git checkout deliberately.
- Import/update the template database through the operator workflow.
- Confirm the intended UUID and revision are available to users.

MISP reads definitions under app/files/misp-objects/objects. Updating the Git submodule alone does not establish database catalogue state. The operator template-update control imports templates and relationships. Admin updateObjectTemplates takes a user ID and reads disk; it does not fetch Git. Keep teaching-only templates away from unattended upstream updates.

Unknown templates and revision upgrades

- Preserve the received attributes and template metadata.
- Normal UI editing needs a suitable definition.
- Review recovery/upgrade mappings and unmatched fields.

Unknown templates and revision upgrades
- Preserve the received attributes and template metadata.
- Normal UI editing needs a suitable definition.
- Review recovery/upgrade mappings and unmatched fields.

The current controller has a recovery/force-update path when normal editing cannot locate the original definition. It is not accurate to claim objects with unknown templates can never be modified through any path. Recovery is not automatically safe: review transferable fields, preserve unmatched data and verify references before saving.

A controlled MISP lab

- Use an unpublished organisation-only event and an authorised writer.
- Add the official object, inspect fields and save.
- Link instances and export/read back the event.
- Test conformity through the actual UI/API path and clean up.

A controlled MISP lab
- Use an unpublished organisation-only event and an authorised writer.
- Add the official object, inspect fields and save.
- Link instances and export/read back the event.
- Test conformity through the actual UI/API path and clean up.

The README describes this optional server exercise. No new live-server or screenshot validation is claimed in this update. The reviewed add API accepts event ID, template UUID and version; object restSearch is also available. Use the REST API course for authentication, TLS, request envelopes and ordinary-user boundaries.

Evolve a model without losing its identity

- Existing model: retain UUID and increment version.
- New model: fresh UUID and initial version.
- Document changed semantics, requirements and consumer impact.

Evolve a model without losing its identity
- Existing model: retain UUID and increment version.
- New model: fresh UUID and initial version.
- Document changed semantics, requirements and consumer impact.

Installing a new template revision does not silently migrate every object instance. A required field added to a model can invalidate older instances under the new definition. Renaming a relation or changing its type can break producers and consumers. Discuss a new model rather than reusing identity for incompatible meanings.

Contribute to the current repository

- Explain why existing templates do not cover the use case.
- Provide schema-valid JSON and synthetic positive/negative instances.
- Check formatting, UUID uniqueness and relationship consistency.
- Open a focused PR with compatibility and validation evidence.

Contribute to the current repository
- Explain why existing templates do not cover the use case.
- Provide schema-valid JSON and synthetic positive/negative instances.
- Check formatting, UUID uniqueness and relationship consistency.
- Open a focused PR with compatibility and validation evidence.

Definitions belong in objects/name/definition.json. Optional icons are sibling icon/icon.svg files, not arbitrary JSON properties. The schema currently includes threat-hunting as a meta-category; read the schema rather than copying an outdated enumeration from prose. The teaching template is not submitted upstream by this training PR.

Follow the upstream validation workflow

```
./jq_all_the_things.sh
# Review changes and commit normalised JSON.
./validate_all.sh
```


Follow the upstream validation workflow
./jq_all_the_things.sh
Review changes and commit normalised JSON.
./validate_all.sh

Use a disposable or dedicated contribution checkout with jq, moreutils/sponge, uuidparse and Python jsonschema installed. Normalisation rewrites JSON across the repository. validate_all.sh checks a clean tree, schema validity, file modes, relationship opposites/tests and UUID uniqueness. Do not mistake the exercise verifier for the full upstream CI suite.

Make the result reviewable

- Show the model, an instance and the intended relationship direction.
- State the exact template and software revisions tested.
- Explain validation limits and migration choices.
- Keep real credentials and incident data out of examples.

Make the result reviewable
- Show the model, an instance and the intended relationship direction.
- State the exact template and software revisions tested.
- Explain validation limits and migration choices.
- Keep real credentials and incident data out of examples.

The course README pins MISP 2.5.50 and misp-objects 999ec328. Validation covers all 454 official definitions against the schema and the supplied offline exercises. It does not claim full upstream CI or live-server testing. The original deck, screenshots and published PDF remain available in the archive, outside standard training.